```
    val source: Source[ByteString, Future[akka.stream.IOResult]] =
      FileIO.fromPath(
        fs.getPath("./chapter14/src/test/resources/characters.csv"))   ❶

    val filterAndGetValues:
         Flow[Map[String, String], Seq[String], NotUsed] =             ❷
      Flow[Map[String, String]]
        .filter(eachMap => eachMap("age").toInt > 18)
        .map(eachMap => eachMap.values.toSeq)                          ❸

    source
      .via(CsvParsing.lineScanner())
      .via(CsvToMap.toMapAsStrings())
      .via(filterAndGetValues)
      .via(CsvFormatting.format())                                     ❹
      .runWith(FileIO.toPath(                                          ❺
        fs.getPath("./chapter14/src/test/resources/result.csv")))
  }
  ...
}
```

❶ Reads a CSV file

❷ Filters the data and gets values

❸ toSeq is needed to transform collection.Iterable to collection.immutable.Iterable.

❹ Formats the lines into a single ByteString

❺ Writes to a CSV file

Here you parse the initial `ByteString` as in the previous example with `CsvParsing.lineScanner()` and create a map from `CsvToMap.toMapAsStrings()`. Then you apply the `filterAndGetValues` function and get back the needed `ByteString` with `CsvFormatting.format()`.

With `CsvFormatting.format()`, you can convert the `Seq[String]` you got from `.filterAndGetValues` into a `ByteString`. This is necessary to write `.toPath`, but it also has some nice extras. You can choose the separator between the columns, the quote character, the end of line, and more.